

Rhobots in Space

Programming the pure rho calculus in immersive space on Apple Vision Pro

Specification v0.6 — draft for alignment

L.G. Meredith

F1R3FLY.io

September 2026

Abstract

Rhobots is an immersive environment in which a programmer builds and runs terms of the pure rho calculus by arranging robots that exchange payloads. The design rests on one property of rho: reduction happens only in the top-level parallel composition. The depth axis can therefore carry the continuation of processing, while scale carries quotation. This note fixes the syntax-to-space mapping, the animation of reduction, and the runtime that decides which reductions occur — an RSpace-based evaluator whose races are settled by the device’s physical cores under a scheduler — together with the control panel through which the programmer sets the pacing mode and the rendering parameters. Construction gestures are sketched only. Full flr3lang is out of scope, but §10 reserves the vocabulary it will need.

Contents

1	Purpose and scope	2
2	The calculus being rendered	2
3	Spatial semantics: two orthogonal axes	3
3.1	Fingerprints	4
4	The rhobot vocabulary	4
4.1	Layout as a function	4
4.2	Handedness	5
4.3	Tethering cables	5
4.4	Binders and scope	6
4.5	Free names: scope behind the programmer	6
5	Dynamics	7
5.1	Motion on the live plane	7
5.2	Keeping cables untangled	8
5.3	The communication animation	8
5.4	Substitution	9
5.5	Structural congruence in the scene	9
5.6	Races, the evaluator, and the scheduler	10
6	Worked examples	10

7	The control panel	11
7.1	Panel layout	11
7.2	Pacing toggle	11
7.3	Core-count slider	13
7.4	Sliders	13
8	Interaction sketch	13
9	Runtime architecture	14
10	Reserved vocabulary for flr3lang	15

List of Figures

1	Depth and scale	4
2	The rhobot vocabulary	5
3	Free names bound behind the programmer	7
4	One communication	8
5	Wandering and approach on the live plane	9
6	Keeping tethering cables untangled	10
7	Replication in pure rho	11
8	Layout of the control panel	12
9	Runtime components	15

1 Purpose and scope

Version 1 renders and executes the pure rho calculus of §2 and nothing else: no ground values, no **new**, no patterns beyond a binding variable, no **where** clauses. The point of this document is agreement. It commits to three things:

1. a total mapping from terms to arrangements of rhobots (§3–§4);
2. the animation of the single reduction rule, including substitution (§5);
3. the runtime contract between the evaluator and the scene (§9).

The control panel is specified in §7; the remaining interaction design (§8) is a sketch to be settled on-device.

2 The calculus being rendered

There are no primitive names; every name is a quoted process.

P, Q	$::=$	0	nil
		for ($y \leftarrow x$) P	receive
		$x!(P)$	send
		$P \mid Q$	parallel
		$*x$	unquote
x, y	$::=$	$@P \mid$ variable	name

Equivalences. Structural congruence $\equiv$ is the least congruence making $(|, \mathbf{0})$ a commutative monoid and including α -equivalence. Name equivalence satisfies $@P \equiv_N @Q$ whenever $P \equiv Q$, and $@*x \equiv_N x$.

Reduction. There is one rule, applied only at top level, closed under parallel composition and structural congruence:

$$\frac{x_0 \equiv_N x_1}{\text{for}(y \leftarrow x_0)P \mid x_1!(Q) \rightarrow P\{@Q/y\}} \text{COMM} \qquad \frac{P \rightarrow P'}{P \mid R \rightarrow P' \mid R} \text{PAR}$$

$$\frac{P \equiv P' \quad P' \rightarrow Q' \quad Q' \equiv Q}{P \rightarrow Q} \text{EQUIV}$$

There is no rule for reduction under a receive prefix or inside a quote.

Free names and the enclosing context. A term the programmer builds may have free name variables $\text{fn}(P) = \{y_1, \dots, y_n\}$. Rhobots never runs an open term. It runs the closure

$$C[P] = \text{for}(y_1 \leftarrow x_1) \cdots \text{for}(y_n \leftarrow x_n)P$$

where each x_i is the channel of a for-comprehension the programmer has written for y_i or, where none was written, the **stdin** channel of a canonically synthesised comprehension (§4.5).

Substitution. $P\{@Q/y\}$ replaces every free occurrence of y in P , including occurrences inside quoted payloads. At a process position the result is $(*y)\{@Q/y\} = Q$ directly; the term $*@Q$ never appears as an intermediate state. This choice is what licenses the immediate unshrink of §5.4.

3 Spatial semantics: two orthogonal axes

The scene has two independent structural axes (Figure 1).

Depth carries guardedness. The plane nearest the programmer, z_0 , holds exactly the top-level parallel composition — the only robots that can act. If a receive stands at depth z_k , its continuation stands at z_{k+1} , directly behind it. Depth increases away from the viewer. In RealityKit coordinates this is the $-z$ direction, a sign that must be fixed once in the scene root and never inverted locally.

Scale carries quotation. A process rendered at scale s appears inside a quote at scale σs for a fixed shrink factor $\sigma < 1$. The recommended starting value is $\sigma = 0.3$, to be tuned on-device. Quoted content is always enclosed in a capsule, so quotation is marked by more than size.

Orthogonality. Depth never changes the scale parameter, and scale never changes depth. Distant robots do subtend smaller visual angles, but that is perspective, not quotation; the capsule disambiguates the two cases.

Elision. When $\sigma^k s$ falls below a legibility threshold $s_{\min}$, the quoted subterm is drawn as an elision glyph: a capsule containing an ellipsis. The glyph carries the subterm’s fingerprint (§3.1), so elided names remain comparable. Inspecting an elided or shrunken term is a viewing operation that opens a magnified inspector. It must never resemble the unquote animation.

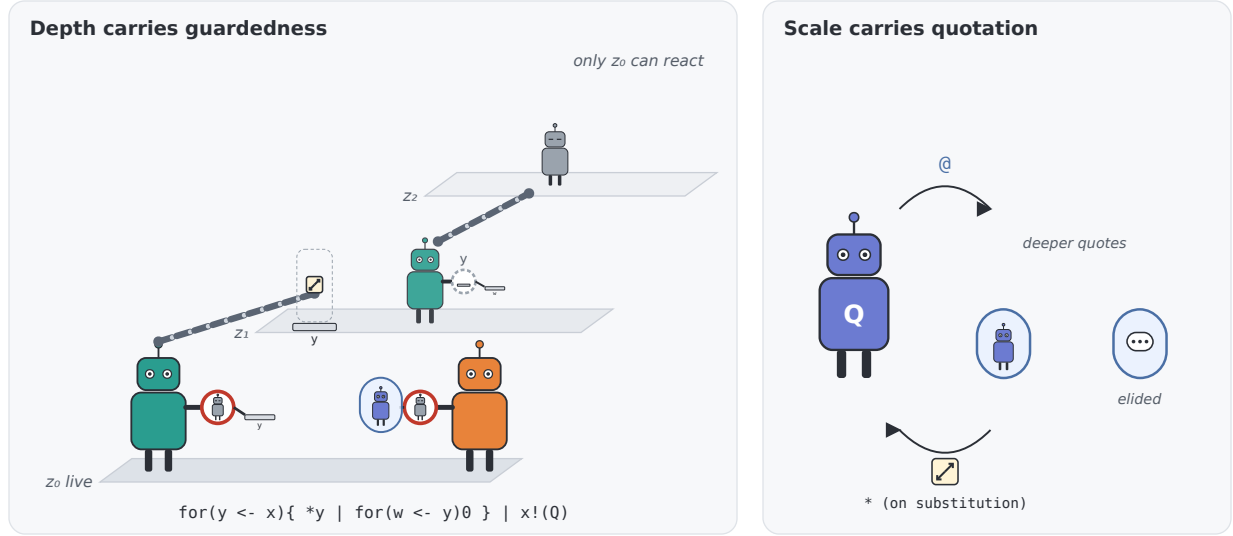


Figure 1: Depth and scale. Left: only robots on the live plane z_0 can react; a receive's continuation stands behind it, one plane per prefix. Right: each quotation shrinks a robot one level into a capsule; below legibility an elision glyph stands in. Unquote, triggered by substitution, restores full size.

3.1 Fingerprints

Every name is rendered with a fingerprint: a colour ring, and optionally a small pattern, derived from a hash of the canonical form of the quoted process, where the canonical form is taken modulo $\equiv$. Two names satisfy $x \equiv_N x'$ exactly when their canonical forms coincide, so matching fingerprints are the visual witness that a communication is possible. Hash collisions are resolved by the evaluator, never by the eye. The fingerprint is a cue, not the test.

4 The rrobot vocabulary

Table 1 and Figure 2 give the mapping.

4.1 Layout as a function

Rendering is a function $\mathcal{L}(P, z, s)$ producing a set of placed robots, where z is the plane and s the scale:

$$\begin{aligned}
 \mathcal{L}(\mathbf{0}, z, s) &= \{\text{inert robot at } (z, s)\} \\
 \mathcal{L}(P \mid Q, z, s) &= \mathcal{L}(P, z, s) \uplus \mathcal{L}(Q, z, s) \quad (\text{packed along } x) \\
 \mathcal{L}(x!(Q), z, s) &= \{\text{sender at } (z, s) \text{ with glyph } \mathcal{G}(x, s), \text{ capsule } \mathcal{L}(Q, 0, \sigma s)\} \\
 \mathcal{L}(\text{for}(y \leftarrow x)P, z, s) &= \{\text{waiter at } (z, s) \text{ with glyph } \mathcal{G}(x, s), \text{ binder } y\} \uplus \mathcal{L}(P, z+1, s) \\
 \mathcal{L}(*y, z, s) &= \{\text{platform } y \text{ with unshrink icon at } (z, s)\}
 \end{aligned}$$

Here $\mathcal{G}(@P, s) = \mathcal{L}(P, 0, \sigma s)$, drawn as the elision glyph below $s_{\min}$, and $\mathcal{G}(y, s)$ is the glyph-slot platform. The layout has two properties to note.

- Depth inside a capsule is local: a receive inside a payload lays out its continuation behind it at the capsule's scale, starting from the capsule's own plane 0.

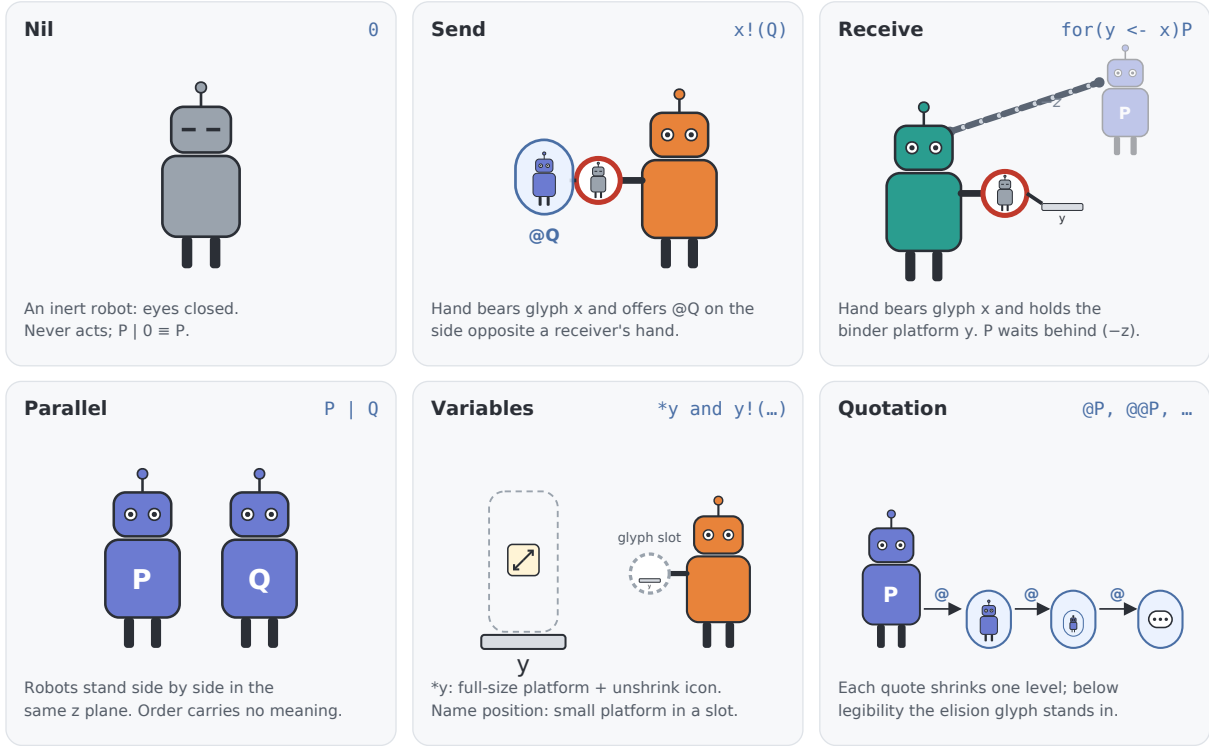


Figure 2: The six elements of pure rho as rhobots.

- $\mathcal{L}$ respects $\equiv$ up to rearrangement along x , and up to optional removal of inert robots (§5.5).

4.2 Handedness

Waiting robots extend their appendage to one side and sending robots to the other. In every figure the waiting robot's hand points right and the sending robot's hand points left, so a waiting robot and a sending robot placed left-to-right face each other, glyph to glyph, with the binder platform and the capsule in the gap between them. Handedness is a fixed convention of the scene. When a committed pair approaches its meeting point (§5.1), the waiting robot always takes the left-hand position, so a communication about to happen reads as two robots arriving face to face.

4.3 Tethering cables

Every waiting robot is joined to its continuation by a tethering cable.

- **Shape.** The cable runs from the back of the waiting robot to a hub on the next plane; light leads fan out from the hub to each top-level component of the continuation. Nested continuations have their own cables, plane by plane, and cables inside capsules are drawn at the capsule's scale.
- **Construction.** A cable is a chain of magnetically linked segments. It is visually distinct from the thin, tinted binder tethers of the next subsection, which appear only on gaze.
- **Following.** A continuation follows its waiting robot with a damped lag rather than rigidly, so a wandering robot visibly drags its future behind it. The lag is what makes cables able to cross (§5.2).

Term	Rhobot	Rendering
0	inert robot	Grey, eyes closed. Never acts.
$x!(Q)$	sending robot	One appendage, on the side opposite to a waiting robot's. The hand bears the channel glyph x and offers a capsule holding Q at scale σ . No continuation stands behind it.
$\text{for}(y \leftarrow x)P$	waiting robot	One appendage in two parts: a hand bearing the channel glyph x , and a held platform labelled y awaiting the payload. P stands one plane behind, joined to the robot by a tethering cable.
$P \mid Q$	adjacency	The robots for P and Q stand side by side on the same plane. Left-to-right order carries no meaning.
$@P$	capsule	The robot for P at scale σ , enclosed. As a channel it is mounted on a hand as the glyph.
$*y$	platform + unshrink icon	A full-size platform labelled y beneath a dashed silhouette, marked with the unshrink icon.
y (name position)	glyph slot	A small platform labelled y sitting in the glyph slot of a hand, or in a capsule, where a shrunken robot will stand.

Table 1: Syntax to space.

- **On communication.** During the advance phase the cable reels in as the continuation moves forward. The hub dissolves when its components arrive on z_0 .
- **Context comprehensions.** The cables of the comprehensions behind the programmer (§4.5) run past the programmer to the working plane. They are shown only when the programmer pulls back.

4.4 Binders and scope

The binder platform on a waiting robot's appendage and every platform labelled y in its continuation are one variable. The scene tints them identically. On gaze or hover it draws tethers from the binder to each occurrence, which makes shadowing visible: an inner binder of the same name breaks the tether chain.

4.5 Free names: scope behind the programmer

Scope is represented with the same device as continuation (Figure 3). A receive's continuation stands in front of it along $-z$, so a term with free names is one the programmer is already standing inside. Free variables mean the programmer is deeper on the z axis, within the continuation of the for-comprehensions that bind them.

- The binding comprehensions stand on context planes $z_{-1}, z_{-2}, \dots$ behind the programmer. The layout function extends to negative depth: $\mathcal{L}(C[P], -n, s)$ places the n enclosing waiting robots on $z_{-n}, \dots, z_{-1}$ and P from z_0 forward.
- The programmer pulls back — by stepping back, or with a dolly gesture — to see the outer for-comprehensions that will receive the free names, tethered to their occurrences on z_0 in the usual binder tint.
- If the programmer wrote an outer comprehension, it is shown as written. If not, it is synthesised canonically: a for-comprehension on the **stdin** channel receives the free name, one comprehension per name, nested in the order the names were first introduced. A synthesised comprehension's hand bears the reserved **stdin** glyph.

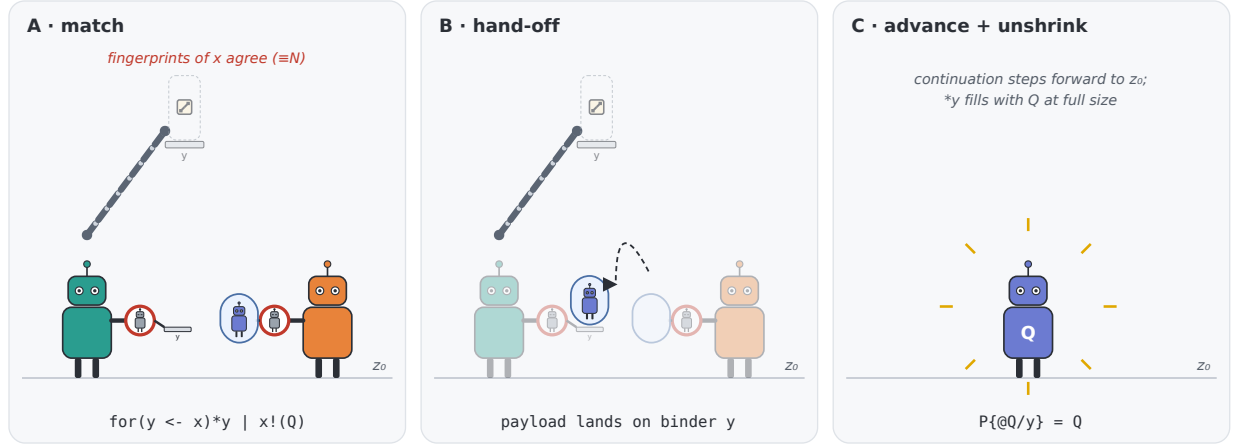


Figure 4: One communication. (A) The glyphs on the two hands carry the same fingerprint. (B) The payload capsule travels to the binder platform while both robots dissolve. (C) The continuation advances to z_0 and every $*y$ platform fills with Q at full size.

5.2 Keeping cables untangled

Cables must never be seen knotted, since a tangle would suggest a relationship between processes that does not exist. Two mechanisms combine (Figure 6).

Avoidance. The wander planner treats every cable as an obstacle. A candidate wander step is rejected if it would bring its robot's cable, or its continuation's lagging path, within a clearance distance of another cable. The planner then picks a different heading. In effect wandering robots keep to lanes and do not pass behind one another.

Break and rejoin. Some crossings cannot be avoided, and these must never be delayed:

- a committed pair approaching its meeting point;
- a robot the programmer drags;
- a plane so crowded that no admissible wander step exists.

In these cases one cable parts at the crossing point: the two neighbouring segments separate and the other cable passes through the gap. The segments then snap back together within a fraction of a second. The cable that parts is the one whose robot is not moving; between two moving robots, a committed approach keeps its cable intact.

A parted cable is a rendering event only. The continuation remains bound to its prefix throughout. The separated ends keep the cable's colour and stay visibly aligned, so a break never reads as a detached continuation.

The control panel offers two cable policies. *Avoid, then break* (the default) applies both mechanisms. *Break freely* disables avoidance, so robots wander without regard to cables and every crossing is resolved by breaking and rejoining.

5.3 The communication animation

A committed COMM between a waiting robot R and a sending robot S plays in four phases (Figure 4):

1. **Approach.** R and S leave their wander and converge on a meeting point (§5.1).
2. **Match.** The two glyphs pulse in their shared fingerprint colour.

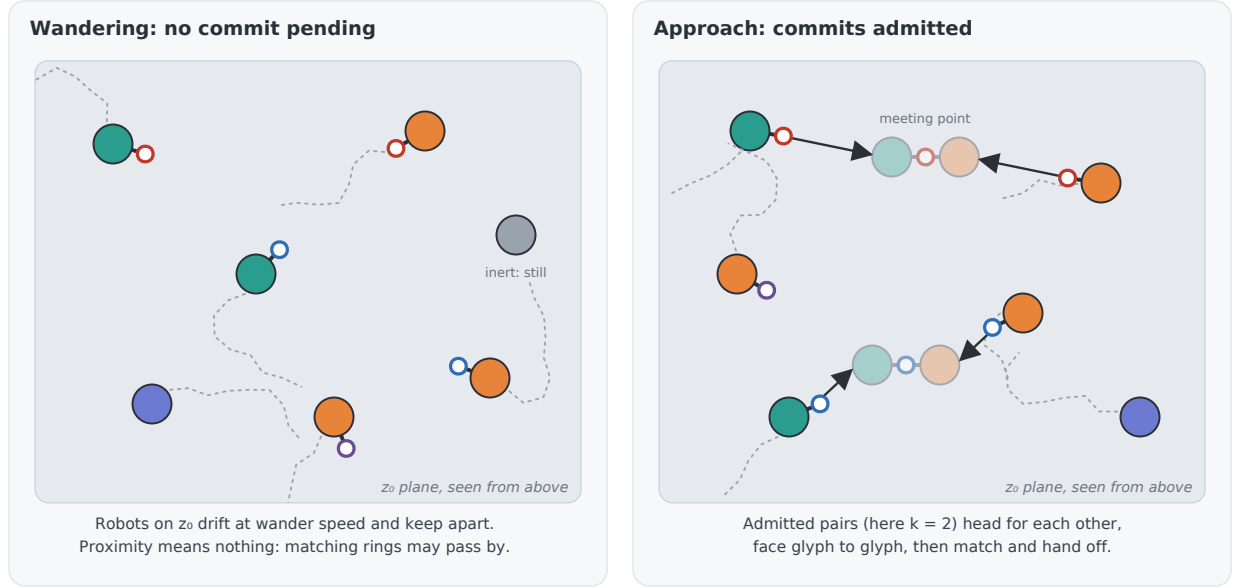


Figure 5: Motion on z_0 , seen from above. Left: with no commit pending, top-level robots wander; nearness carries no meaning. Right: once commits are admitted, each committed pair heads for a shared meeting point and arrives face to face, while every other robot keeps wandering.

3. **Hand-off.** The capsule crosses the gap between the facing hands, from S 's hand to R 's binder platform. R and S fade out: a send has no continuation, and a receive is consumed.
4. **Advance and substitute.** Everything behind R moves forward one plane, so its first plane becomes part of z_0 . Substitution then fills the platforms (§5.4).

The approach lasts as long as the distance requires at the approach speed. The remaining three phases take the communication duration. Both are set from the control panel (§7).

5.4 Substitution

Substitution of $@Q$ for y fills every platform labelled y in the advanced continuation, according to the occurrence's position.

- **Process position** ($*y$): the payload unshrinks immediately to the platform's own scale. The platform and its unshrink icon disappear, leaving the robot for Q . Unshrinking is relative: a $*y$ inside a capsule at scale σs restores Q to σs , not to s .
- **Name position** (glyph slot or capsule slot): the shrunk Q takes the slot at the slot's scale and stays shrunk. Its fingerprint appears, so the robot's channel becomes comparable with others.

5.5 Structural congruence in the scene

Positions on a plane carry no meaning: robots wander (§5.1), and the programmer may also move them by gesture. Inert robots produced by reduction fade after a delay set on the control panel, applying $P \mid \mathbf{0} \equiv P$, unless the programmer pins them. Pinning is a viewing preference with no semantic effect.

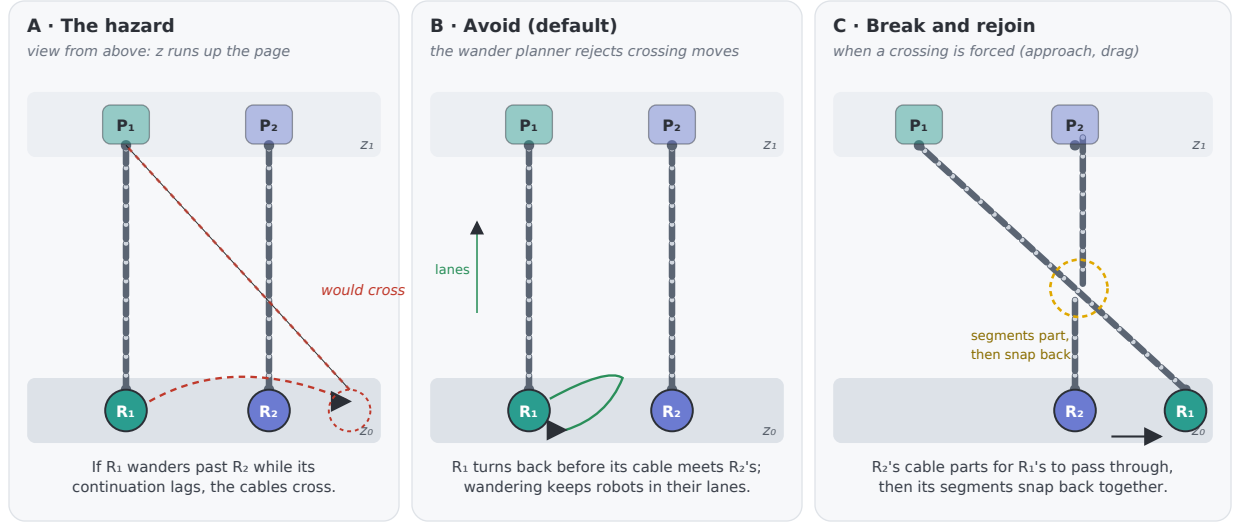


Figure 6: Tethering cables seen from above, with z running up the page. (A) A robot wandering past a neighbour while its continuation lags would cross cables. (B) By default the wander planner steers away from any move that brings two cables within clearance. (C) When a crossing is forced, the crossed cable parts at the crossing point, lets the other through, and its segments snap back together.

5.6 Races, the evaluator, and the scheduler

Several COMM redexes may be available at once, and they may contend for the same sender or receiver. Rhobots does not ask the programmer to pick. Races are decided by an RSpace-based rho evaluator running on the device's physical cores, mediated by its scheduler (§9). The scene is a consumer of committed events.

- The evaluator emits an ordered *commit log*. Each entry records the identities of the consumed receive and send, the channel's canonical key, and the substitution performed.
- The scene animates entries in log order. How evaluation and animation are paced against each other is the programmer's choice, set by the pacing toggle (§7.2).
- When a committed communication consumes a robot that other redexes also wanted, the losing partners briefly show their unmatched glyphs, so the race is visible after the fact.

6 Worked examples

A single communication. The term $\text{for}(y \leftarrow x)*y \mid x!(Q)$ reduces to Q , as shown in Figure 4.

Replication from reflection. Let $D = \text{for}(y \leftarrow x)(x!(*)y \mid *y)$. Then

$$x!(D \mid P) \mid D \rightarrow x!(D \mid P) \mid D \mid P,$$

so the pair $x!(D \mid P) \mid D$ behaves as $!P$. In the scene (Figure 7), a sender hands D a shrunken copy of D itself together with P . Two substitutions happen in the continuation. The $*y$ inside the new sender's capsule unshrinks relative to that capsule, rebuilding the capsule's contents. The full-size $*y$ releases $D \mid P$ onto z_0 . The pattern re-forms on the live plane with one more P . This example is the first acceptance test: if it is legible, the metaphor is sound.

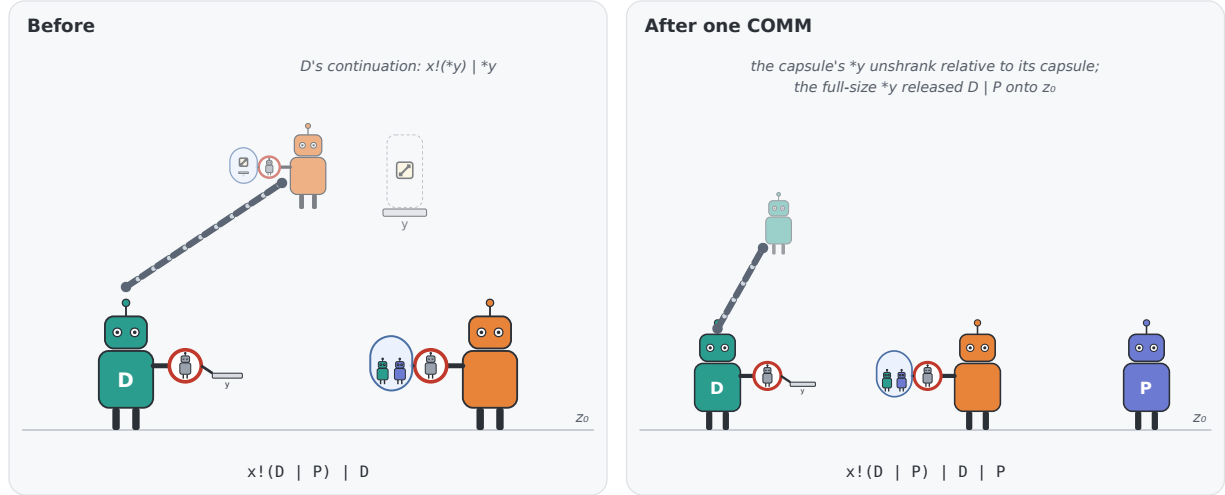


Figure 7: Replication in pure rho. After one communication the sender–waiter pair re-forms and a copy of P joins the live plane.

7 The control panel

Controls that tune the environment rather than edit the program are collected on one floating panel. The panel is a visionOS window placed at or above eye level, beside the live plane rather than in front of it. It can be dismissed and recalled, and it persists its settings between sessions. Changing any control never changes the term being run.

7.1 Panel layout

Figure 8 shows the panel. Its controls fall into five groups, from top to bottom.

Group	Controls	Specified in
Execution	Run, Pause, Step round; Lockstep / Free-running toggle	§7.2
Concurrency	Worker-thread slider (1 to the cores visible to the container); throughput read-out	§7.3
Playback	Communication duration; playback buffer size; buffer fill and admission status	§7.2
Motion	Wander speed; approach speed; cable-tangle policy	§5.1, §5.2
Rendering	Shrink factor σ ; legibility threshold $s_{\min}$; plane spacing; inert fade delay	§3

7.2 Pacing toggle

A two-state toggle sets how the scene and the evaluator are paced.

Lockstep. The scheduler admits a round of at most k commits, where k is the core-count setting (§7.3). It then withholds admission of the next round until the scene acknowledges that every animation in the round has finished. The pairs in a round approach their meeting points together. Worker threads still race on the enabled cores to reach each commit, so races are still decided by the cores; only admission is gated. The scene therefore shows each round’s communications simultaneously, exactly as they happened, at human pace.

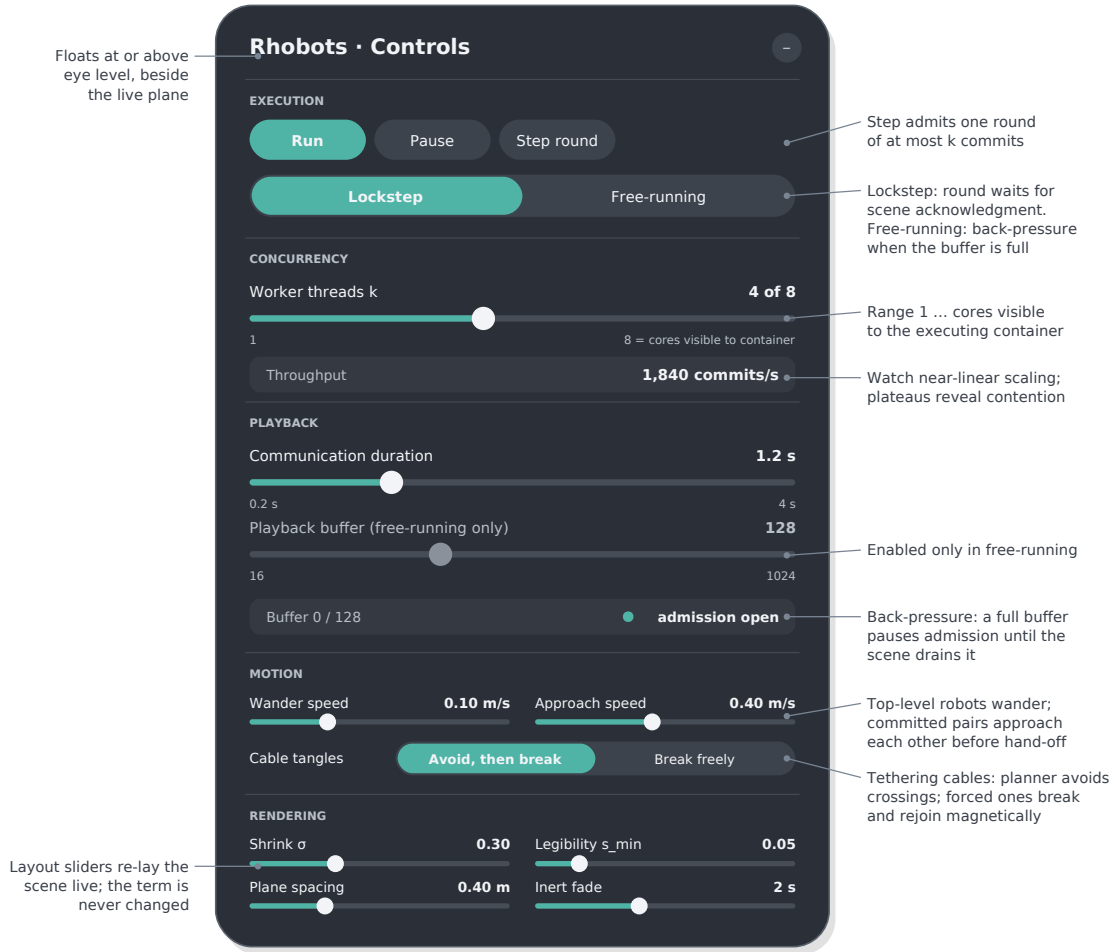


Figure 8: Layout of the control panel. Five groups: execution (run controls and the pacing toggle); concurrency (the worker-thread slider, from one core to the cores visible to the executing container, with its throughput readout); playback (communication duration, the buffer-size slider, and the buffer’s back-pressure status); motion (wander and approach speeds, and the cable-tangle policy); and rendering (shrink factor, legibility threshold, plane spacing, inert fade).

Free-running. The evaluator commits without waiting for acknowledgments. The commit log feeds a bounded playback buffer that the scene drains at the animation rate. The buffer applies back-pressure: when it is full, the scheduler pauses admission until the scene has drained an entry, so the evaluator never runs further ahead of the scene than the buffer bound and every commit is animated. A robot named in any buffered entry is already consumed. It keeps wandering until its entry is released to the scene, and is never shown approaching anyone other than its committed partner.

Switching from free-running to lockstep pauses admission until the buffer drains, then continues in lockstep. The panel shows the buffer’s fill and whether admission is open or paused by back-pressure. Switching from lockstep to free-running takes effect at the next admission. Run, pause and single-step controls sit beside the toggle; single-step admits exactly one round in either mode.

7.3 Core-count slider

A slider sets the number of worker threads k from 1 to the number of cores that appear to the executing container as physical cores — its available parallelism as reported to the process, which may be fewer than the device’s hardware cores. It is the principal tool for getting a feel for races and for scaling.

- With $k = 1$, commits are strictly interleaved: the scene never shows two communications in progress at once.
- With k cores, at most k communications are ever in progress at the same time. Rounds in lockstep hold at most k commits, and in free-running mode no more than k animations overlap.
- A throughput readout beside the slider shows commits per second, so the programmer can move the slider and watch throughput rise roughly linearly with k .

The rise is linear only while redexes lie on distinct channels. Communications contending for the same channel serialise on that channel’s lock whatever k is, and the readout will show the plateau. That is itself a lesson the slider teaches. Changing k takes effect at the next admission.

7.4 Sliders

Each rendering parameter has a slider. Ranges and defaults are starting points to be tuned on-device.

Sliders that affect layout (shrink factor, threshold, plane spacing) re-lay the scene smoothly while it runs, since they change only the rendering function $\mathcal{L}$ and never the term.

8 Interaction sketch

The scene is a visionOS `ImmersiveSpace`. Hand tracking is available only there.

Device constraints carried over from the F1R3Skein bring-up.

- Selectable controls sit at or above eye level, because eye tracking was unreliable in the lower field.
- The scene root fixes the depth sign once.
- Required Info.plist and entitlement keys, including hand-tracking and world-sensing usage descriptions, are part of the build checklist from day one.

Parameter	Range	Default	Effect
Shrink factor σ	0.15–0.6	0.3	Scale of a quoted process relative to its container (§3).
Legibility threshold $s_{\min}$	0.02–0.2	0.05	Scale below which a quote is drawn as the elision glyph.
Plane spacing	0.15–1.0 m	0.4 m	Distance between successive depth planes.
Communication duration	0.2–4 s	1.2 s	Time of match, hand-off and advance (§5).
Wander speed	0–0.3 m/s	0.1 m/s	Drift speed of top-level robots; 0 stills the plane (§5.1).
Approach speed	0.1–1.5 m/s	0.4 m/s	Speed at which a committed pair converges (§5.1).
Cable tangles	two-state	avoid, then break	Cable policy (§5.2).
Inert fade delay	0 s–never	2 s	How long inert robots remain before fading; “never” keeps them.
Playback buffer	16–1024 commits	128	Back-pressure bound in free-running mode; disabled in lockstep.
Worker threads k	1–visible cores	visible cores	Concurrency width of the evaluator (§7.3).

Intent	Gesture (provisional)
Create a robot	Pinch a nil, send or waiting robot from a palette floating at eye level.
Parallel composition	Drop a robot beside others on a plane.
Set a continuation	Drop a robot onto the region behind a waiting robot.
Quote	Drop a robot into a sender’s capsule or onto a hand’s glyph slot; it shrinks as it enters.
Bind / use a variable	Label the binder platform; pinch it to spawn a <i>*y</i> platform or a glyph-slot platform tethered to it.
Inspect a quote	Look at a capsule and pinch-and-spread to open a magnified inspector; the capsule’s contents are not changed.
Run / pause / step	On the control panel (§7).
Walk into a continuation	Double-pinch a waiting robot to make its continuation temporarily frontmost, for inspection only.

9 Runtime architecture

Evaluator. The evaluator is an RSpace over pure rho (Figure 9).

- A top-level send *produces* its payload at its channel’s canonical key.
- A top-level receive *consumes* at its key.
- A match commits atomically and yields the continuation with the substitution applied.
- Its top-level parallel components are then fed back as new produces and consumes.
- Worker threads, k of them as set on the control panel, race on keys under striped locks. The scheduler owns thread assignment and commit admission, and is the single place where scheduling policy lives. In lockstep mode admission waits for the scene’s acknowledgment; in free-running mode it does not (§7.2).

Relation to f1r3node-rust (feature/cost-accounted-rho). The node supplies the reference implementation and possibly reusable parts.

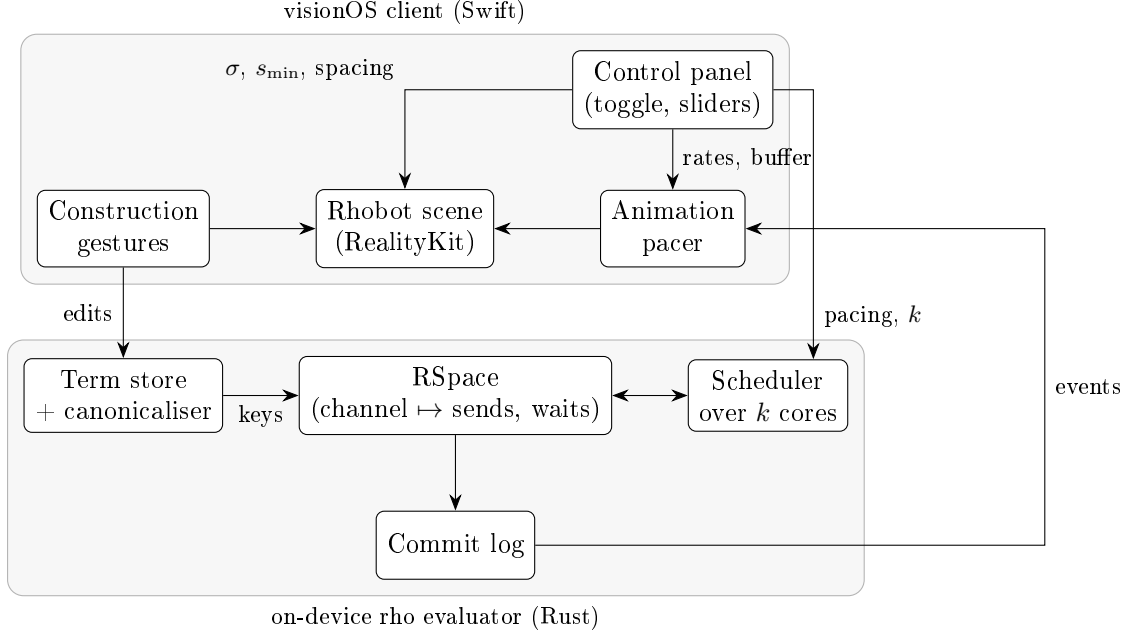


Figure 9: Runtime components. The scene edits terms; the evaluator decides and logs communications; the scene animates the log. The control panel sets rendering parameters, playback rates, and the pacing mode, which the scheduler enforces as admission control.

- **rspace++** provides the tuple-space core, matcher and striped locks. Its hot store is in memory; its history layer depends on LMDB through **heed**. Version 1 needs only the hot store. Building for the visionOS Rust target is to be verified.
- The node’s **Par** normal form and sorter provide canonical ordering. Pure rho maps into the node’s grammar through **Nil**, **x!(P)**, **for(y <- x){P}**, **|**, ***x** and **@P**. This allows differential testing of the on-device evaluator against the node.
- The **rho-pure-eval** crate evaluates guard expressions for **where** clauses. Despite its name it is not a process reducer and is not the evaluator described here.
- Deployment of a constructed term to a shard, and cost accounting on the scene, are deferred.

The stdin channel. The evaluator reserves one channel key for **stdin**. The scene’s input gesture becomes a produce on that key, carrying the quoted process the programmer placed. For differential testing against the node, **stdin** corresponds to a system channel rather than to any quoted process.

Transport. Version 1 runs the evaluator in-process on the headset, linked as a static library behind a C ABI. This avoids the device-boundary transport failures met during the F1R3Skein bring-up.

10 Reserved vocabulary for f1r3lang

f1r3lang construct	Reserved rrobot form
Patterns in receipts	The binder platform takes the shape of the pattern: a moulded tray with sub-platforms for pattern variables.
Joins (; and &)	One waiting robot with several appendages, one per receipt; all hands must be served for the robot to commit.
where clauses	A visor on the waiting robot through which payloads are inspected before acceptance.
Graded where	The visor shows a meter; the value in the resolution algebra decorates the commit.
new	An enclosure on the plane; channels minted inside carry an unforgeable fingerprint.
Ground values	Tokens rather than robots, carried in capsules.
DDL Theory terms	Distinct robot species per theory, with rewrites as a second kind of animation.
Cost accounting	A charge gauge on each robot, drawn down by the commits it takes part in.